LOGTEL
It's more than knowledge

IOT (and M2M and WoT)

Day 4 - Hands-on lab | 2026

By Samuel Dratwa
Samuel.dratwa@gmail.com

The course: four days, one story

LOGTEL
It's more than knowledge

Day 1
Foundations

- Networking: OSI and TCP/IP
- Wi-Fi
- Wireshark: seeing packets
- Hardware interfaces
- What IoT is - and why now
- Bluetooth, NFC, RFID, UWB

Day 2
Technologies

- Architecture and cloud
- LPWAN, cellular, satellite
- LoRa and LoRaWAN
- Zigbee, Thread, Matter
- MQTT and brokers
- Grafana dashboards
- MakeCode warm-up

Day 3
Cellular IoT and security

- Cellular IoT: NB-IoT, LTE-M
- D2D and V2X
- Device management, OTA updates
- End-to-end IoT security
- Cellular modem practice

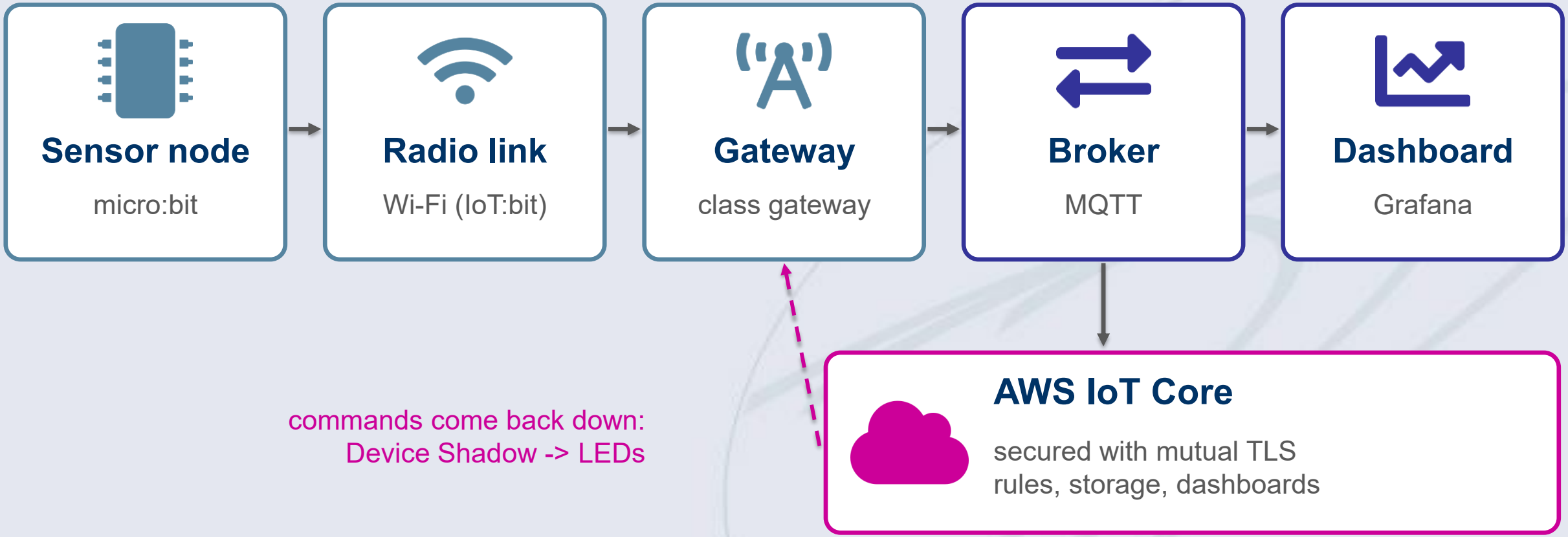
Gilad

Day 4
Hands-on lab

- AWS IoT Core
- Certificates and mTLS
- micro:bit to the cloud
- MQTT + Wireshark + Grafana
- Cloud-to-device control
- Troubleshooting challenges

TODAY

The pipeline we build - one step per day



Day 1 - understand each part

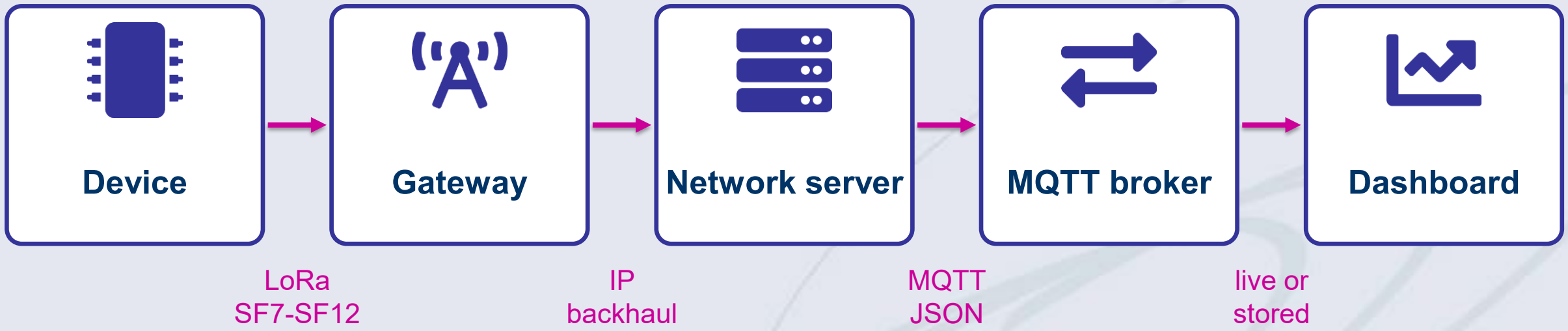
Day 2 - messaging, dashboards

Day 4 - build it end to end

Day 4 - Hands-on lab: today's plan

1. **Lab overview**
   the chain we build today
2. **AWS IoT Core**
   broker, shadow, rules
3. **Certificates, mTLS**
   how a device proves who it is
4. **Part 1: micro:bit**
   IoT:bit, Wi-Fi, class gateway
5. **Parts 2-3: identity**
   console, your own certificate
6. **Parts 4-5: control**
   shadow, rules, class wall
7. **Part 6: break + fix**
   troubleshooting challenges
8. **Wrap-up**
   the course in one picture

Day 2 in one picture



- LoRaWAN: small, rare messages over kilometres
- MQTT: publish to a topic - the broker delivers to every subscriber
- Grafana: a live view now; a database for history and alerts

Today: we build this chain ourselves - micro:bit to AWS IoT Core

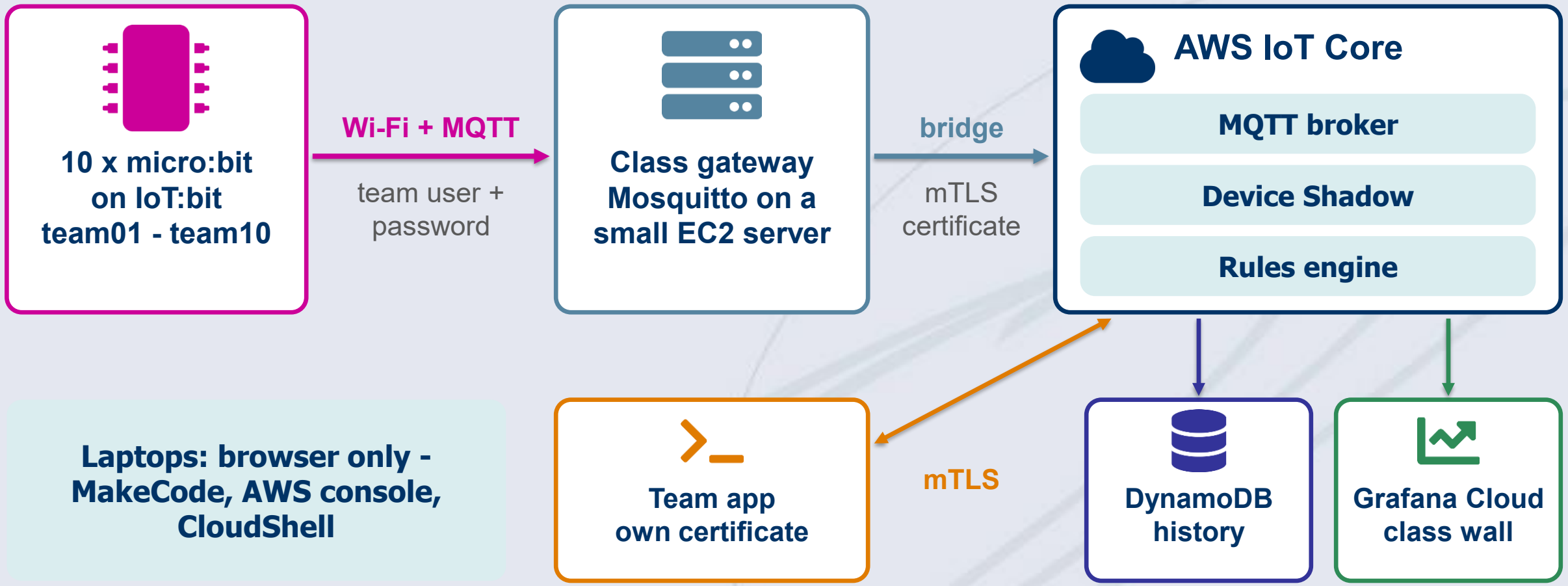
Day 2 quiz - the answers

1	SF7 or SF12: farther, and the cost?	**SF12 - but about 25x the airtime**
2	Which LoRaWAN activation, and why?	**OTAA: fresh session keys at every join**
3	site/+/door matches site/haifa/pump-1/door?	**No: + is exactly one level**
4	QoS levels on AWS IoT Core?	**0 and 1 only - no QoS 2**
5	A device died silently - how do we know?	**Keep-alive + Last Will message**
6	Why can't the Grafana MQTT plugin alert?	**Live stream only - nothing is stored**

THE LAB AT A GLANCE

One chain, ten teams, six parts

Today's lab: the whole chain



Commands come back the same way: shadow -> rule -> lab/teamNN/cmd -> micro:bit

Your team's kit

On the table

- micro:bit V2 on its IoT:bit board
- USB cable
- A laptop - browser only

On your card

- Gateway user and password
- AWS console user
- Wi-Fi: see the whiteboard

Your number

- Team 01 to 10
- Topics lab/teamNN/...
- Thing teamNN

Example team card (placeholders)

```
TEAM 03
Wi-Fi     <Logtel network name>       password: on the whiteboard
Gateway   <gateway address>  port 1883   user team03   password <on the card>
AWS       <console sign-in link>   user team03   Region N. Virginia (us-east-1)
```

Six parts, one chain

#	Part	Time	Done when ...
1	micro:bit to the gateway	60 min	your line moves on the class wall
2	AWS console tour	20 min	your messages scroll in the MQTT test client
3	Your own certificate	45 min	your team app prints your readings
4	Shadow and rules	40 min	a heart appears; DynamoDB keeps history
5	The class wall	15 min	ten teams on one Grafana dashboard
6	Break and fix	40 min	you name the cause from the logs

Do not move on until the check works - every part builds on the one before

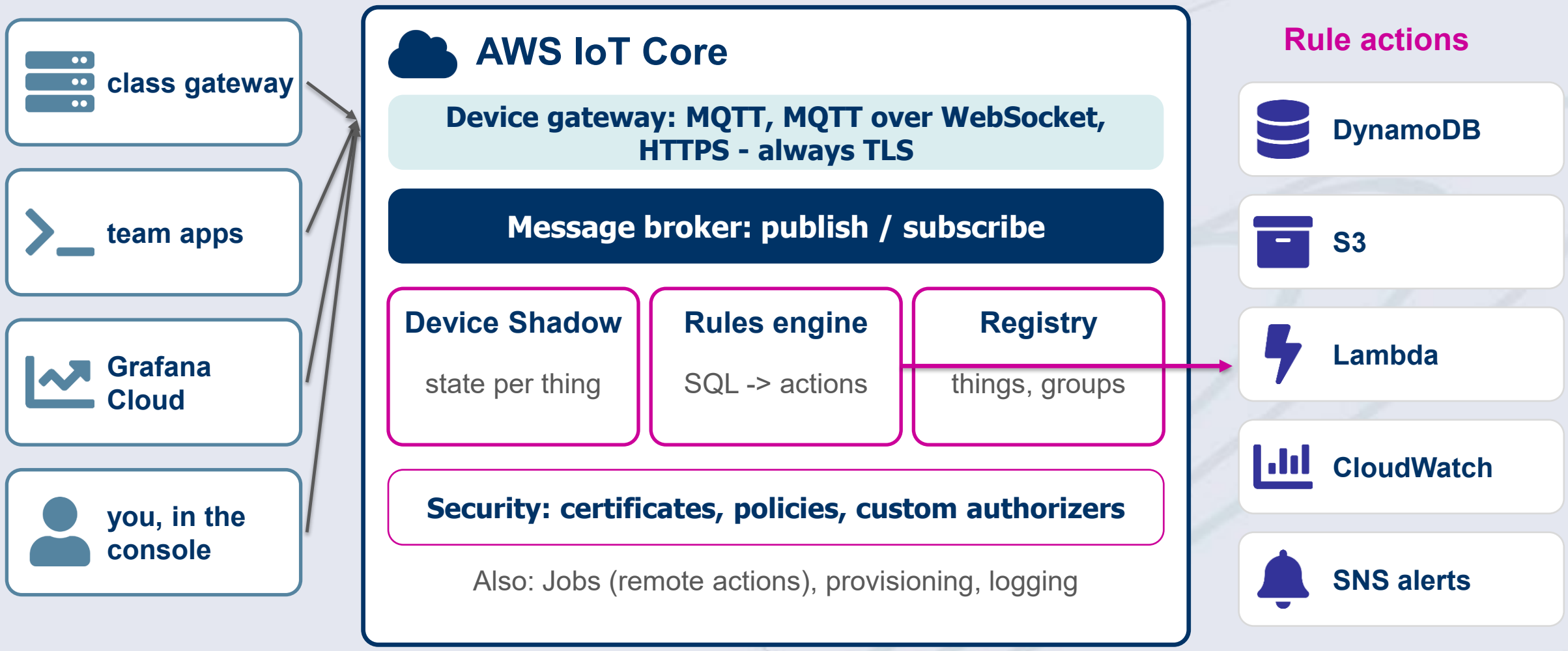
Lab rules

- Use only your own team number: topics, client ID, thing
- A private key stays in its file: no chat, no e-mail, no screenshots
- Do not change or delete what another team created
- Stuck for five minutes? Check the troubleshooting slide, then call us
- Everything is deleted after the course - certificates included

AWS IOT CORE

Managed broker, shadow, rules and security

AWS IoT Core: the big picture



How clients connect

Protocol	Port	Proves identity with	Used today by
MQTT over TLS	8883	X.509 certificate	gateway, team apps, Grafana
MQTT over TLS	443 + ALPN	X.509 certificate	if a firewall blocks 8883
MQTT over WebSocket	443	AWS login (SigV4)	the console's test client
HTTPS	443 / 8443	X.509 certificate	publish only

TLS 1.2 or 1.3 - always

MQTT 3.1.1 and MQTT 5

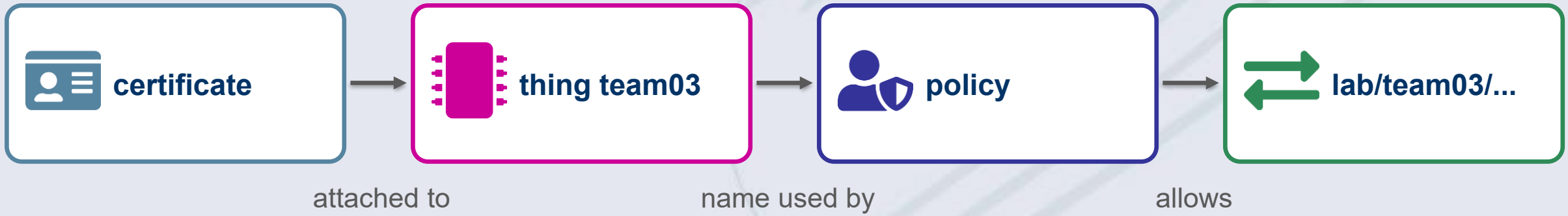
QoS 0 and 1 - no QoS 2

The endpoint name is per account: xxxx-ats.iot.us-east-1.amazonaws.com

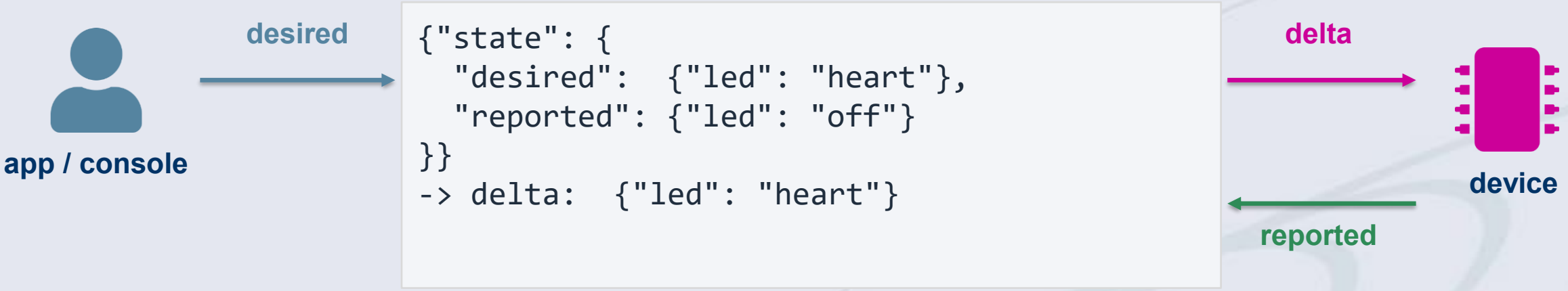
Things: the device registry

```
{
  "thingName": "team03",
  "attributes": {
    "room": "logtel-lab",
    "board": "iot-bit"
  }
}
```

- A thing = a name plus attributes, groups and types
- Not needed to connect - but it ties a certificate to a name
- AWS advice: thing name = MQTT client ID
- Our lab: team01 ... team10



Device Shadow: desired, reported, delta



- desired = what we want; reported = what the device says; delta = the difference
- A device that was offline gets the delta when it reconnects
- Topics: $aws/things/team03/shadow/update, .../update/delta, .../get

Our lab: the micro:bit only speaks lab/... - a rule copies the delta to lab/team03/cmd

Rules engine: SQL for messages

```
SELECT temp, light, topic(2) AS team, timestamp() AS ts
FROM 'lab/+/telemetry'
WHERE temp > 30
```

FROM which topics - wildcards allowed

SELECT which fields, renamed, plus functions such as topic() and timestamp()

WHERE only the messages that match - optional

Action where the result goes: DynamoDB, S3, Lambda, SNS, Republish ...

Every matching message runs the rule - no server of our own, no polling

Rule actions: where the data goes

LOGTEL
It's more than knowledge

Store

DynamoDB
S3
OpenSearch
InfluxDB, Timestream

Stream

Kinesis Data Streams
Firehose
Apache Kafka

Compute

Lambda
Step Functions

Notify

SNS
SQS
CloudWatch alarms

Observe

CloudWatch metrics
CloudWatch Logs

Other

HTTP
Republish
IoT SiteWise
Location

Gone: IoT Analytics (Dec 2025) and IoT Events (May 2026) - older tutorials still show them

Limits that matter in class

Limit	Value	What it means for us
Publishes per connection	100 per second	our bridge carries all ten teams: 2 per second
Connects per client ID	1 per second	a fast reconnect loop gets throttled
Same client ID twice	older one dropped	every team app needs its own ID
Payload size	128 KB	our JSON is about 30 bytes
Topic	256 bytes, 7 slashes	lab/team03/telemetry is fine
Keep-alive	30 to 1200 s	shorter values become 30 s

Quotas are per account and Region - our whole class is far below all of them

What does today cost?

43,200	$0.17	$0.20	$0
messages: 10 teams, every 5 s, 6 h	AWS IoT Core + DynamoDB	gateway server, 24 hours	CloudShell, Grafana Cloud free tier

- AWS IoT Core: $1.00 per million messages, $0.15 per million rules and actions (us-east-1)
- Each subscriber that receives a message is billed again - the class wall doubles the count
- Real fleets: the cost driver is messages x subscribers, not the number of devices

Less than one dollar for the whole lab day

The AWS IoT console: where things are

Console menu	What is there	Part
Test > MQTT test client	watch and send messages	2, 4, 6
Manage > All devices > Things	team01 ... team10 and their shadows	3, 4
Security > Certificates, Policies	your certificate, the team policy	3, 6
Message routing > Rules	telemetry -> DynamoDB, delta -> cmd	4
CloudShell (top bar icon)	a Linux terminal in the browser	3
CloudWatch > Log groups	AWSIotLogsV2 - who failed and why	6

Top right: Region must be N. Virginia (us-east-1) - otherwise you see nothing

CERTIFICATES AND MTLS

How a device proves who it is - mutual TLS

Why not just a password?

Password

- The same secret travels at every login
- The server stores it - one leak exposes many devices
- Easy to copy to another device

Certificate + private key

- The private key never leaves the device
- The server keeps only public information
- Each device can be blocked on its own

In our lab both appear: the micro:bit logs in to the class gateway with a **password**; the gateway, your app and Grafana prove themselves to AWS with **certificates**.

An X.509 certificate is an ID card

X.509 certificate (example)

Subject	CN=AWS IoT Certificate
Issuer	the AWS IoT CA
Valid	2026-10-13 to 2049-12-31
Public key	RSA, 2048 bits
Serial	4f:3a:...:9c
Signature	made with the issuer's key

- **Subject** - whose card it is
- **Issuer** - who issued it, like the country on a passport
- **Valid** - the expiry date
- **Public key** - the 'photo' to check the holder against
- **Signature** - the stamp nobody can forge

The certificate is public. The private key is the secret.

Proving identity, keeping the secret



Public key

inside the certificate - anyone may see it

Private key

a file on the device - never sent anywhere

Who vouches for whom?

LOGTEL
It's more than knowledge

Your app checks AWS

Amazon Root CA 1

AmazonRootCA1.pem - you download it

↓ signs

intermediate CA

sent by the server

↓ signs

***.iot.us-east-1.amazonaws.com**

the server's certificate

AWS checks your app

AWS IoT CA

or your own registered CA

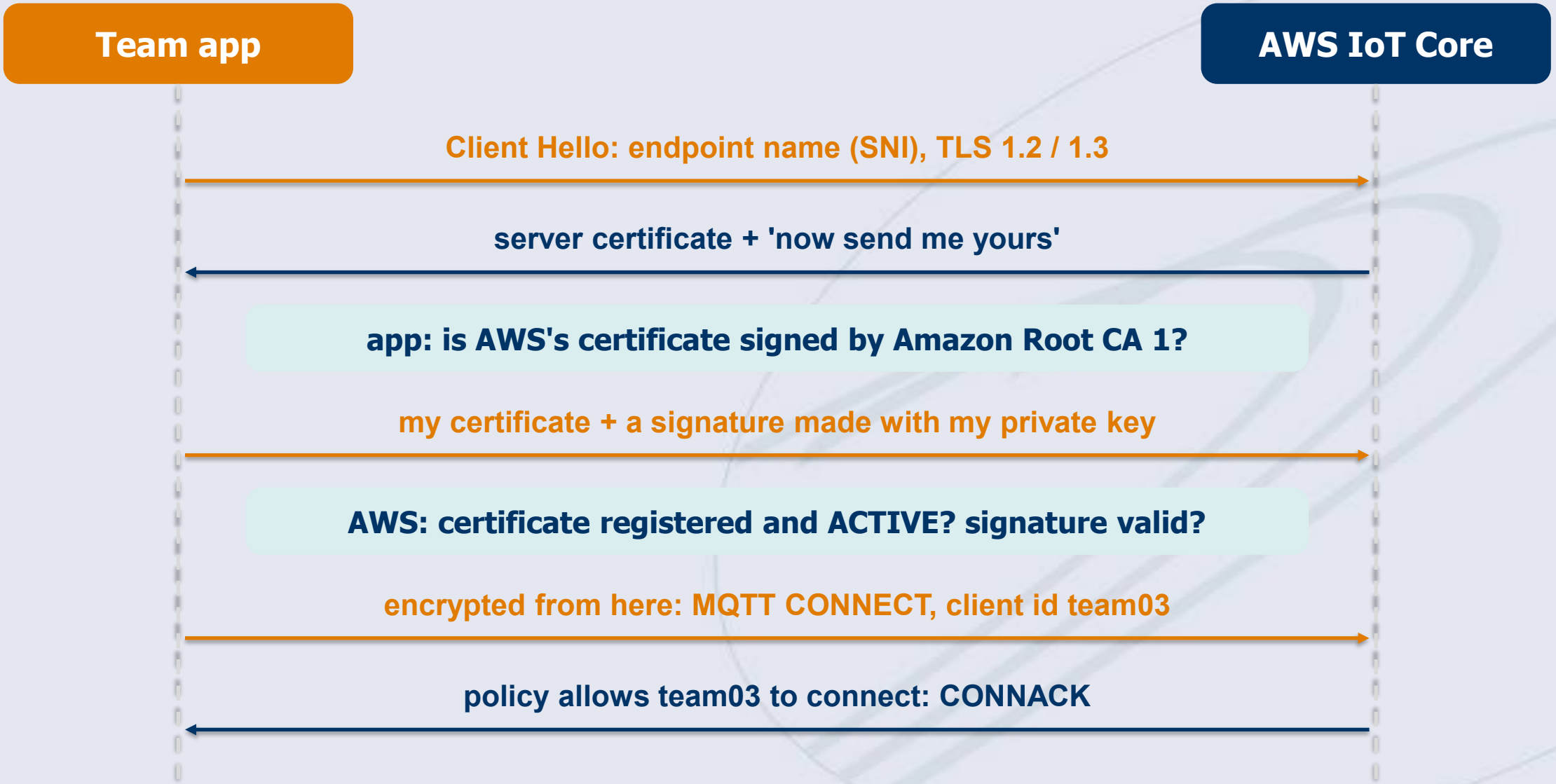
↓ signs

team03 certificate

registered in AWS IoT, ACTIVE

Missing or wrong CA file = 'certificate verify failed' - the most common lab error

Mutual TLS, step by step



From certificate to permission

```
{"Effect": "Allow", "Action": "iot:Connect",
 "Resource": "...:client/${iot:Connection.Thing.ThingName}"}
{"Effect": "Allow", "Action": ["iot:Publish", "iot:Receive"],
 "Resource": "...:topic/lab/${iot:Connection.Thing.ThingName}/*"}
{"Effect": "Allow", "Action": "iot:Subscribe",
 "Resource": "...:topicfilter/lab/${iot:Connection.Thing.ThingName}/*"}
```

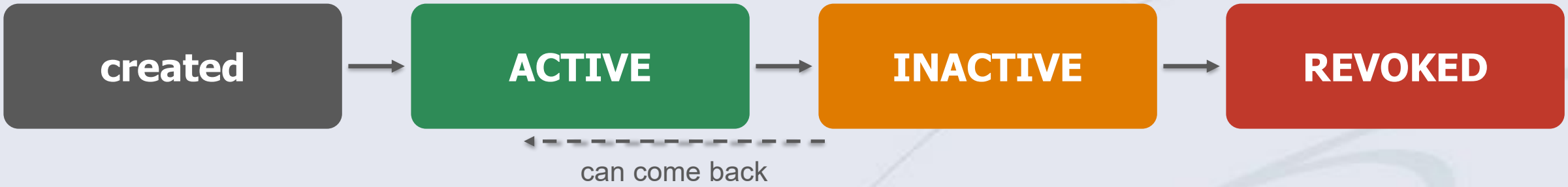
- Everything is denied unless a policy allows it
- ${iot:Connection.Thing.ThingName} = the thing attached to the certificate
- One policy for all ten teams - each one reaches only lab/<its name>/...
- Client ID must equal the thing name; max 2048 characters, up to 10 policies per certificate

Three ways to get a certificate

Method	Who makes the private key	Good for
One-click (console or CLI)	AWS - download it once!	labs, prototypes - our lab
Certificate signing request	the device; it never leaves	careful production devices
Fleet provisioning by claim	the device, at first connection	factories, thousands of units
Your own CA (JITP / JITR)	your factory or HSM	fleets with their own PKI

One-click shows the private key ONCE. Lose it, and the only fix is a new certificate.

A certificate's life



- Only an ACTIVE certificate can connect - new ones need activating
- Lost or stolen device: set its certificate INACTIVE at once
- AWS-signed certificates last until 31 Dec 2049 - your own CA chooses its dates
- Rotation = new certificate first, then retire the old one (AWS IoT Jobs can automate it)

Part 6 challenge: one team's certificate is suddenly INACTIVE

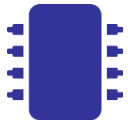
Where should a private key live?



A file on a computer — our lab: CloudShell and the gateway server



Encrypted storage on the device — better - but the processor still sees the key



Secure element or TPM chip — the key is born in the chip and never leaves it



Module with built-in cloud identity — e.g. AWS IoT ExpressLink modules

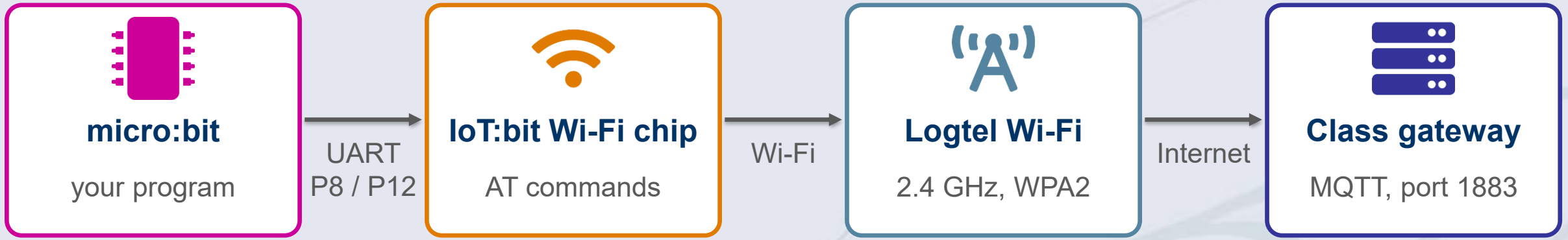
weaker → stronger

Recall Day 3 with Gilad: secure boot, crypto elements, end-to-end security

HANDS-ON: SIX PARTS

micro:bit, gateway, AWS IoT Core, dashboard

Part 1: micro:bit to the class gateway



1. Open the lab template in MakeCode (link on your card)
2. Set your team number, the Wi-Fi password (whiteboard) and your gateway password
3. Download to the micro:bit; power the IoT:bit from its own USB socket
4. Wait for the tick on the LEDs - then find your line on the class wall

Done when **your temperature appears on the projected class wall**

Part 1a: Wi-Fi first, then MQTT

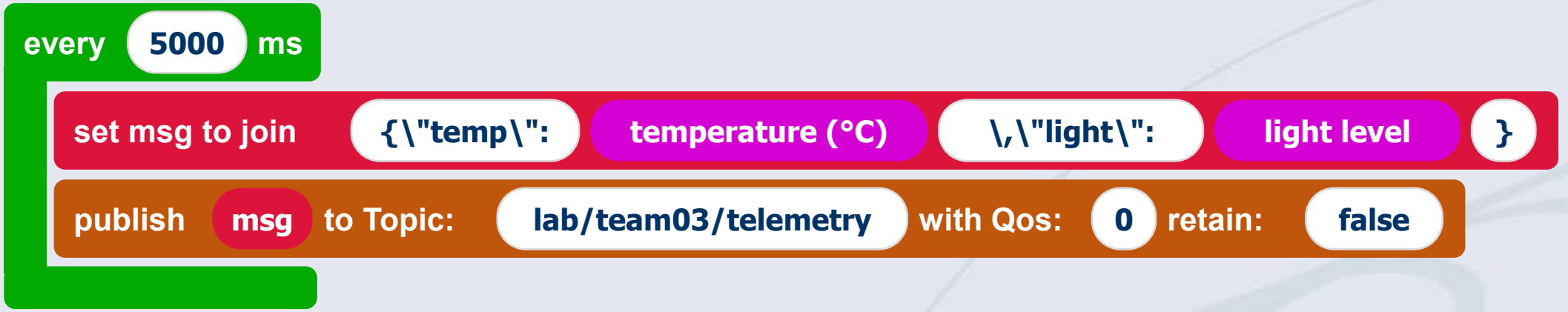
```
on start
    set ESP8266 RX [P8] TX [P12] Baud rate [115200]
    connect Wifi SSID = [<Logtel Wi-Fi>] KEY = [<password>]
    Set MQTT client config scheme: [TCP] clientID: [team03]
        username: [team03] password: [<on your card>] path: [ ]
    connect MQTT broker host: [<gateway>] port: [1883] reconnect: [false]
    if [MQTT broker is connected] then
        show icon [yes]
    else
        show icon [sad]
```

Wi-Fi: 2.4 GHz; password on the whiteboard

IDs: client ID = user name = team03

Port 1883: plain MQTT - a lab shortcut

Part 1b: publish every 5 seconds



Why the backslashes? The message travels inside an AT command:

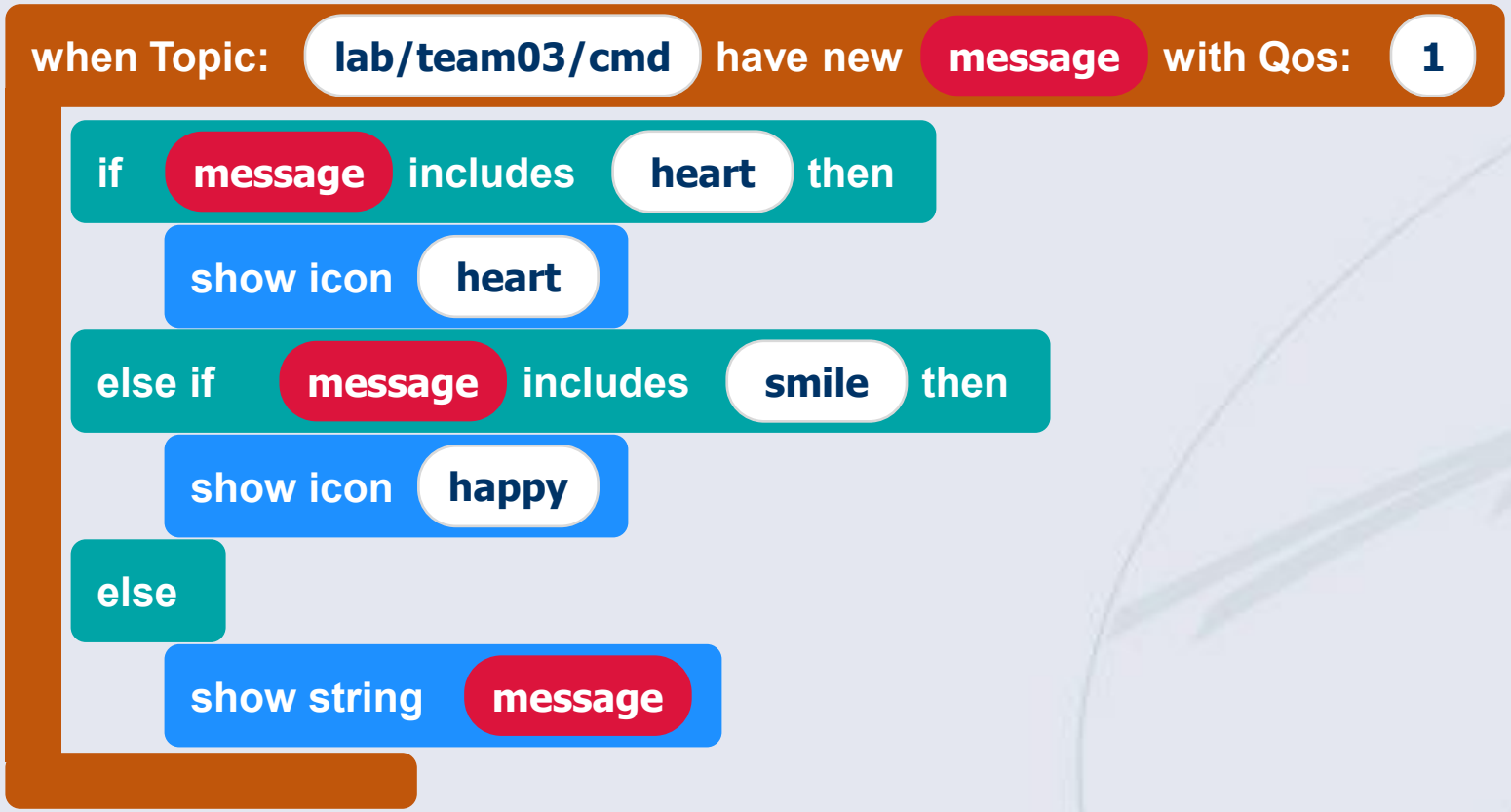
```
AT+MQTTPUB=0,"lab/team03/telemetry","{\"temp\":23\,\"light\":120}",0,0
```

↓

```
{"temp":23,"light":120}
```

A plain " or , would end the text early - so they travel as \" and \,

Part 1c: react to commands



One word

a command with a comma is cut at the first comma

Exact topic

no wildcards in the topic box

Who sends?

the test client (2), your app (3), the shadow (4)

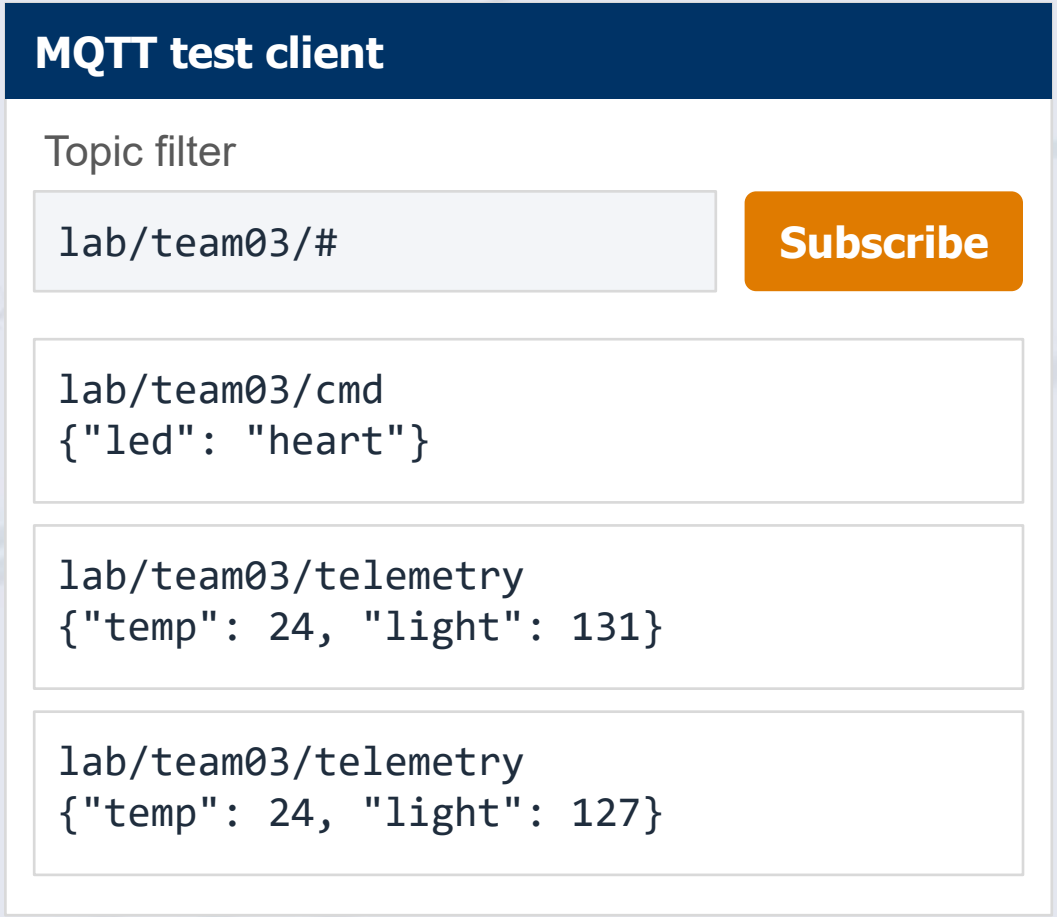
When the micro:bit does not connect

What you see	Check
Nothing on the LEDs	program downloaded? IoT:bit powered from its own USB?
Sad face	Wi-Fi name and password; gateway address, port, team password
Sad face on every board	gateway down, or the network blocks port 1883
Tick, nothing on the wall	topic spelling; the backslashes in the message
Restarts or freezes	power the IoT:bit from its own USB socket
Another team's board drops out	two boards with the same client ID

Same method as always: the lowest broken layer first

Part 2: find your data in AWS

1. Sign in with the link and user on your card
2. Region, top right: N. Virginia (us-east-1)
3. IoT Core > Test > MQTT test client
4. Subscribe to lab/team03/#
5. Watch your readings arrive every 5 s
6. Publish {"led": "heart"} to lab/team03/cmd



Done when your readings scroll in the test client and the heart appears on the micro:bit

Wireshark: what the network still sees

1. Start a capture on the laptop's network card
2. Reload the MQTT test client page
3. Filter: tls.handshake.type == 1
4. Open a Client Hello - find the server name
5. Everything after it: Application Data

```
Transport Layer Security
  Handshake: Client Hello
    Extension: server_name
      Server Name:
      xxxx-ats.iot.us-east-1
      .amazonaws.com
TLSv1.3 Record Layer
  Application Data
    3f9c0a61d4e2...
```

The network sees whom you talk to - not what you say

Day 2 reminder: plain MQTT on port 1883 shows user, password and data

Part 3: your own certificate



- Your app gets the identity team03 - the same name as your thing and its topics
- It talks to AWS IoT Core directly with mutual TLS - no gateway in between
- CloudShell = a Linux terminal inside the AWS console: nothing to install

Done when your team app prints the readings and 'heart' changes the LEDs

In CloudShell: create and attach

```
T=team03
ARN=$(aws iot create-keys-and-certificate --set-as-active \
  --certificate-pem-outfile $T.cert.pem --private-key-outfile $T.private.key \
  --query certificateArn --output text)
aws iot attach-policy --policy-name lab-team-policy --target $ARN
aws iot attach-thing-principal --thing-name $T --principal $ARN
curl -sO https://www.amazontrust.com/repository/AmazonRootCA1.pem
EP=$(aws iot describe-endpoint --endpoint-type iot:Data-ATS --output text)
```

- Open CloudShell from the >_ icon in the console's top bar - same Region
- Paste one block at a time; if CloudShell restarts, set T and EP again (the files stay)

Done when your .pem and .key files are there, and echo $EP prints the endpoint

Run your team app

```
R=https://raw.githubusercontent.com/SADOT-SD/IOTlab/main
curl -fsSLO $R/team_app.py
pip3 install --user paho-mqtt
python3 team_app.py $T $EP
```

It prints your readings - type a word to send a command:

```
connected to xxxx-ats.iot.us-east-1.amazonaws.com:8883 as team03
lab/team03/telemetry  {"temp": 24, "light": 131}
heart
sent to lab/team03/cmd: heart
```

Try it: type team04 heart
what happens, and why?

One copy per team: a second copy with the same name throws out the first

Done when your readings appear in CloudShell and 'heart' changes the LEDs

Check your identity

```
openssl x509 -in team03.cert.pem -noout -subject -issuer -dates
subject=CN = AWS IoT Certificate
issuer=...                    <- who signed it? read it here
notAfter=Dec 31 23:59:59 2049 GMT
```

Security > Certificates

your certificate is ACTIVE

Things > team03 > Certificates

attached to your thing

The policy

lab-team-policy is attached

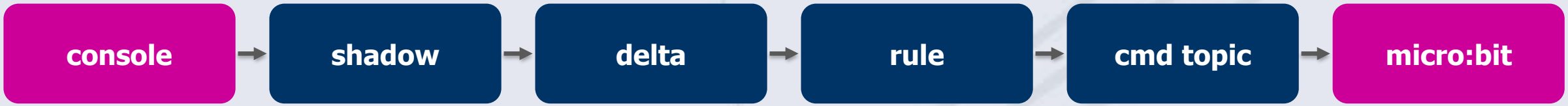
The private key file never leaves CloudShell - it is the team's identity

Done when all three checks are green - you onboarded a client with X.509

Part 4: commands through the shadow

1. Things > team03 > Device Shadows
2. Classic Shadow > Edit
3. Set desired.led and press Update
4. Watch the micro:bit
5. Try smile - then hi

```
{"state": {
  "desired": {"led": "heart"}
}}
```



Done when the micro:bit changes within a few seconds of Update

The two lab rules (ready-made)

lab_telemetry_to_dynamodb

```
SELECT temp, light, topic(2) AS team,
       timestamp() AS ts
FROM 'lab/+/telemetry'
```

-> DynamoDB table lab_telemetry

lab_shadow_to_cmd

```
SELECT state.led AS led
FROM '$aws/things/+/shadow/update/delta'
```

-> Republish to lab/${topic(3)}/cmd

topic(3) of $aws/things/team03/shadow/update/delta is team03

History in DynamoDB

1. DynamoDB > Tables > lab_telemetry
2. Explore items
3. Query: team = team03
4. Sort by ts, newest first

team	ts	temp	light
team03	1792058467123	24	131
team03	1792058462120	24	128
team03	1792058457118	23	126
team03	1792058452115	23	131

- The class wall forgets when you close it - the table remembers
- ts is milliseconds since 1970: rules add the time, the micro:bit does not need a clock

Done when **you find your own rows - and they keep growing**

Part 5: the class wall

MQTT data source (Grafana Cloud)

URI	`tls://xxxx-ats.iot.us-east-1.amazonaws.com:8883`
Client ID	`grafana-wall`
TLS client auth	`on: certificate + key`
Query	`topic lab/team03/telemetry`

- A third identity: the dashboard certificate
- Its policy may only subscribe to lab/+/telemetry
- One panel per team - ten lines on one wall
- Live only: nothing is stored in Grafana

Lecturer demo - the free tier has three active users, so we project it

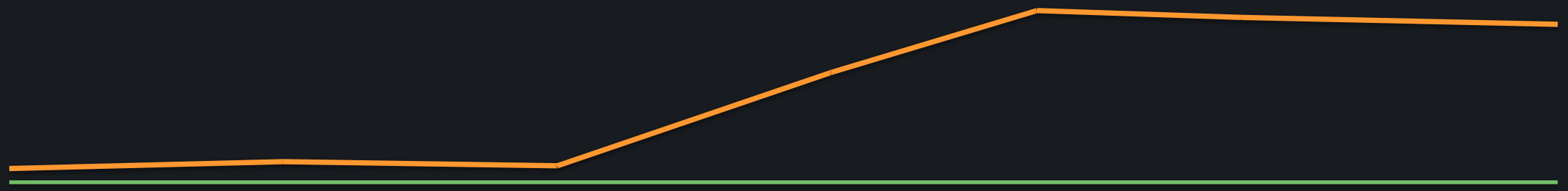
The class wall

Logtel IoT lab - live temperature

team01	team02	team03	team04	team05
23 °C	24 °C	22 °C	25 °C	24 °C

team06	team07	team08	team09	team10
31 °C	23 °C	24 °C	22 °C	24 °C

team06 is warm: someone is holding the micro:bit



TROUBLESHOOTING

Break it, find it, fix it

Find the broken layer - bottom up

Layer	Ask	Look at
Power, board	Is it on? Is our program running?	LEDs, MakeCode
Wi-Fi (L1-2)	Did the IoT:bit join the network?	status icon on the micro:bit
IP, DNS (L3)	Address? Does the name resolve?	nslookup, ping
TCP port (L4)	Can we reach the port at all?	Test-NetConnection, nc
TLS (L5-6)	Do the certificates check out?	openssl s_client, script errors
MQTT (L7)	Connected? Right topic? Right ID?	MQTT test client
Policy (L7)	Is it allowed?	AWSIotLogsV2

Fix the lowest broken layer first - everything above it depends on it

AWS IoT logs in CloudWatch

- Switched on for the whole account (lecturer)
- Log group: AWSIotLogsV2
- Levels: ERROR, WARN, INFO, DEBUG
- Search for your client ID or topic
- Read status and reason first

```
{
  "timestamp": "2026-10-14 11:41:07.123",
  "logLevel": "ERROR",
  "eventType": "Publish-In",
  "clientId": "team03",
  "topicName": "lab/team04/cmd",
  "status": "Failure",
  "reason": "AUTHORIZATION_FAILURE"
}
```

Example entry, shortened: team03 tried another team's topic

```
fields @timestamp, eventType, status, reason
| filter clientId = "team03" | sort @timestamp desc
```

Why was the connection closed?

Reason (lifecycle event)	Meaning	Typical cause in class
AUTH_ERROR	certificate or policy said no	inactive certificate
DUPLICATE_CLIENTID	someone took over your client ID	two apps named team03
MQTT_KEEP_ALIVE_TIMEOUT	silent for 1.5 x keep-alive	Wi-Fi dropped, device froze
CONNECTION_LOST	the network went away	laptop asleep, network changed
THROTTLED	too many requests	a reconnect loop
CLIENT_ERROR	the client broke a protocol rule	e.g. payload too large

`Topic: $aws/events/presence/disconnected/<clientId> - field disconnectReason`

Symptom, cause, fix

Symptom	Likely cause	Fix
micro:bit never joins Wi-Fi	wrong name or password, 5 GHz	whiteboard password; 2.4 GHz
Wall shows nothing for us	topic typo (Lab/ vs lab/)	copy the topic from the card
'certificate verify failed'	wrong or missing CA file	AmazonRootCA1.pem, right path
Closed right after connecting	policy, inactive certificate	AWSIotLogsV2, certificate state
Two apps keep kicking each other	same client ID	one ID per running app
Publish with QoS 2 refused	AWS IoT has no QoS 2	use QoS 0 or 1

Part 6: break-and-fix challenges

LOGTEL
It's more than knowledge

1 Sad face

The micro:bit never reaches the gateway any more

2 Cut off

The team app connects and is dropped at once

3 One-way

Readings arrive, but commands never reach the board

4 Tug of war

Two apps keep disconnecting each other

5 Refused

The team app fails after a 'small improvement'

6 Invisible

Your line is missing from the class wall

For each: which layer? what evidence? what fix? - then tell the class

Where real systems slow down



- The slowest link sets the speed of the whole chain
- Send less: packed bytes, report on change, longer intervals (Day 2)
- Measure first: message counts in CloudWatch, gateway load, Wi-Fi retries

Our class: 2 messages per second - a real fleet: thousands

What we built today

LOGTEL
It's more than knowledge

micro:bit + IoT:bit

sensor node on Wi-Fi

Class gateway

passwords in, certificate out

AWS IoT Core

broker, policy per team

Rules

DynamoDB history, commands

Shadow

desired -> LEDs

Class wall

Grafana Cloud, live

The same pattern as LoRaWAN: device -> gateway -> network server -> application

The course in one picture

LOGTEL
It's more than knowledge

Day 1 - the parts

- Layers and protocols
- Wi-Fi, Wireshark
- GPIO, I2C, SPI, UART
- Bluetooth, NFC, UWB

Day 2 - the system

- LoRaWAN and LPWAN
- Zigbee, Thread, Matter
- MQTT in depth
- Grafana dashboards

Day 3 - Gilad

- Cellular IoT
- D2D and V2X
- OTA updates
- End-to-end security

Day 4 - the cloud

- AWS IoT Core
- X.509 and mutual TLS
- Shadow and rules
- Troubleshooting

Device -> gateway -> broker -> rules -> people
the same chain at every scale

Final quiz

1. Why does our micro:bit need the class gateway?
2. In mutual TLS, what really proves identity?
3. Which file lets your app trust AWS?
4. What is the shadow's delta?
5. Cut off right after a publish - where do you look first?
6. Two clients with the same ID - what happens?

Thank You!!!